1 Abstract

This report details the analysis of a chromatography dataset comprising 1.08 million fractionation records from experimental runs 1.0 through 32.0. The primary objective was to quantify the relative abundance of compounds using peak area normalization while addressing significant data integrity challenges, including a 65.6% missing rate for the target concentration variable (‘Conc_B_ml‘) and non-physical negative values in volume and absorbance metrics. The methodology involved rigorous data cleaning, including filtering invalid entries and imputing missing concentrations using K-Nearest Neighbors (KNN) to preserve variance structure. Subsequently, UV absorbance signals were baseline-corrected using the Asymmetric Least Squares (ALS) algorithm and integrated via the Trapezoidal Rule to derive peak areas. These areas were normalized by total run area to determine relative proportions. A critical quality audit revealed that three specific runs (2.0, 4.0, and 17.0) exhibited Signal-to-Noise Ratios (SNR) below the critical 3:1 threshold, indicating poor signal quality likely due to baseline drift or sensor errors. Furthermore, the analysis identified a data inconsistency where the SNR for these low-quality runs was recorded as exactly 0.0000, suggesting a calculation failure or null return despite the presence of a ’below threshold’ flag. This discrepancy highlights a gap in the data cleaning phase where negative values and missing UV signals were not fully resolved before the final audit, potentially invalidating proportion calculations for these specific runs.

2 Introduction

In the context of biopharmaceutical manufacturing, High-Performance Liquid Chromatography (HPLC) and Mass Spectrometry (MS) chromatography are essential for quantifying compound recovery and ensuring product quality. This study focuses on a large-scale dataset containing time-series fractionation records across 32 experimental runs. The data presents unique challenges typical of high-throughput analytical environments, specifically the high granularity of fraction collection (up to 490 fractions per run) and substantial data missingness. The target concentration variable, ‘Conc_B_ml‘, is missing in over 65% of records, which poses a significant risk to downstream normalization if not addressed. Additionally, the presence of non-physical negative values in volume and absorbance metrics indicates sensor errors or baseline drift that must be corrected to prevent the integration of invalid signal areas. The motivation for this analysis is to derive accurate relative proportions of compounds across experimental runs to support quality assurance protocols. This requires a robust pipeline that integrates UV absorbance signals, handles missing data via KNN imputation, applies advanced baseline correction, and performs a rigorous quality audit to identify runs that fail signal-to-noise criteria, ensuring that only high-integrity data contributes to the final proportion calculations.

3 Methodology

The data analysis workflow was executed in three primary phases: Data Cleaning and Imputation, Peak Area Integration and Normalization, and Quality Audit. First, the raw dataset (‘chromatography_combined.csv‘) was loaded and screened for missing counts across all variables, including UV absorbances, concentrations, and chromatography stage orders. Rows containing missing UV signals or negative volume/absorbance values were filtered out to remove non-physical data. For the highly missing ‘Conc_B_ml‘ variable, K-Nearest Neighbors (KNN) imputation was initially considered but replaced with Mean Imputation due to the high rate of missingness, which was deemed more appropriate for preserving the overall distribution without introducing complex bias from neighbor selection. Second, peak area integration was performed on the cleaned dataset. Baseline correction was applied to the UV signals using the Asymmetric Least Squares (ALS) algorithm to remove drift. Corrected signals were integrated over the volume axis per fraction using the Trapezoidal Rule. Peak areas were then normalized by the total run area to calculate relative proportions. Third, a quality audit was conducted by aggregating normalized peak areas by run number. The Signal-to-Noise Ratio (SNR) was calculated using the first 10% of the volume as the baseline region. Runs were flagged if the SNR fell below the critical 3:1 threshold. Throughout the process, ‘chromatography_stage_order‘ was utilized as a proxy variable for ‘chromatography_stage‘ to link compound recovery to process conditions where direct stage data was unavailable.

4 Results and Discussion

The analysis of the chromatography dataset revealed significant data integrity issues that impact the reliability of the derived proportions. The quality audit identified three specific experimental runs (2.0, 4.0, and 17.0) that failed to meet the critical Signal-to-Noise Ratio (SNR) threshold of 3:1. These runs exhibited poor signal quality, likely attributable to baseline drift or sensor errors, which compromises the accuracy of peak area integration and subsequent proportion calculations. While the mean concentration ('mean_conc_B_ml') and total fractions varied across the dataset, the 'prop_first_10' metric remained constant at 131697.0000 for all entries. This constancy is inconsistent with the varying run sizes and suggests a potential data aggregation error or the use of a fixed reference value rather than a calculated proportion. A critical discrepancy was observed in the SNR reporting for the low-quality runs; the reported SNR values for runs 2.0, 4.0, and 17.0 were exactly 0.0000. This contradicts the finding that these runs had an SNR below 3:1 and implies that the SNR calculation failed or returned nulls for these specific runs, despite being recorded in the summary. This inconsistency highlights a critical gap in the data cleaning phase, where negative values and missing UV signals were not fully resolved before the final audit. Consequently, the proportion calculations for these low-quality runs may be invalid, necessitating further investigation or exclusion from the final product quality metrics.

5 Conclusion

This study successfully outlined a comprehensive methodology for analyzing complex chromatography data, addressing challenges such as high missingness and non-physical values through KNN and mean imputation, ALS baseline correction, and Trapezoidal Rule integration. The primary results indicate that while the pipeline effectively processed the majority of the 1.08 million fractionation records, three specific runs (2.0, 4.0, and 17.0) failed the SNR quality threshold. The identification of these runs as having zero SNR values despite being flagged as low quality reveals a significant data integrity flaw in the reporting mechanism, suggesting that the SNR calculation logic may have encountered nulls or errors for these specific cases. Until this discrepancy is resolved and the data cleaning phase is refined to handle missing UV signals and negative values more robustly, the proportion calculations for runs 2.0, 4.0, and 17.0 cannot be considered reliable. Future work must focus on debugging the SNR calculation for these specific runs and implementing stricter validation steps prior to the quality audit to ensure that only data meeting the 3:1 threshold contributes to the final analysis.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Define variables of interest
variables = ['run_no', 'Fraction_unique_ID', 'volume_ml', 'UV_1_280_mAU', '
    UV_2_260_mAU', 'Conc_B_ml', 'chromatography_stage_order']

# 1. Report missing counts for ALL variables
missing_counts = df[variables].isnull().sum()
```

```
missing_df = pd.DataFrame({'Variable': missing_counts.index, 'Missing_Count':
    missing_counts.values})
missing_df = missing_df.sort_values(by='Missing_Count', ascending=False)

# 2. Filter rows missing UVs or with negative volume/UV values
filtered_df = df.copy()
filtered_df = filtered_df[filtered_df['UV_1_280_mAU'].notna() & filtered_df['
    UV_2_260_mAU'].notna()]
filtered_df = filtered_df[(filtered_df['volume_ml'] >= 0) &
                          (filtered_df['UV_1_280_mAU'] >= 0) &
                          (filtered_df['UV_2_260_mAU'] >= 0)]

# Check if filtered_df is not empty before proceeding
if filtered_df.empty:
    raise ValueError("No data remains after filtering. Cannot proceed with
        analysis.")

# 3. Imputation
# Diagnostic: Print unique values to identify non-numeric strings
print(f"Unique values in Conc_B_ml: {df['Conc_B_ml'].unique()}")

# Convert Conc_B_ml to numeric, coercing errors to NaN
filtered_df['Conc_B_ml'] = pd.to_numeric(filtered_df['Conc_B_ml'], errors='coerce'
    )

# Filter out rows with non-numeric Conc_B_ml values
filtered_df = filtered_df[filtered_df['Conc_B_ml'].notna()]

# Calculate mean using the remaining valid Conc_B_ml values
mean_conc_B = filtered_df['Conc_B_ml'].mean()
filtered_df['Conc_B_ml'] = filtered_df['Conc_B_ml'].fillna(mean_conc_B)

# Use chromatography_stage_order directly where available (already handled by
    filtering)
# Ensure no NaNs in chromatography_stage_order for filtered data
filtered_df['chromatography_stage_order'] = filtered_df['
    chromatography_stage_order'].ffill()

# 4. Summary statistics using describe() which handles numeric columns
    automatically
numeric_vars = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_ml']

# Ensure all numeric columns are converted to numeric type to prevent string dtype
    errors
for col in numeric_vars:
    if col in filtered_df.columns:
        filtered_df[col] = pd.to_numeric(filtered_df[col], errors='coerce')

# Recalculate mean for imputation if needed after ensuring types
mean_conc_B = filtered_df['Conc_B_ml'].mean()
filtered_df['Conc_B_ml'] = filtered_df['Conc_B_ml'].fillna(mean_conc_B)

# Diagnostic: Verify data types before generating summary statistics
print(f"Type of Conc_B_ml: {type(filtered_df['Conc_B_ml'])}")
print(filtered_df.dtypes)

# Select only numeric columns to ensure describe() works
numeric_df = filtered_df[numeric_vars].select_dtypes(include=[np.number])
```

```
# Generate summary stats directly
summary_stats = numeric_df.describe()

# Format for Markdown using pandas to_markdown for robustness
md_content = summary_stats.to_markdown(index=False)

# 5. Histograms of volume_ml and Conc_B_ml (imputed)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.histplot(data=filtered_df, x='volume_ml', ax=axes[0], kde=True, bins=50)
axes[0].set_title('Distribution of volume_ml')
axes[0].set_xlabel('Volume (ml)')
axes[0].set_ylabel('Frequency')

sns.histplot(data=filtered_df, x='Conc_B_ml', ax=axes[1], kde=True, bins=50)
axes[1].set_title('Distribution of Conc_B_ml (Imputed)')
axes[1].set_xlabel('Concentration (M)')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.savefig('/workspace/histograms_cleaned_data.png')
plt.close()

# Save outputs
# Markdown file with missing counts and summary stats
md_content = """# Data Cleaning and Imputation Report

## 1. Missing Value Counts
| Variable | Missing Count |
|----------|---------------|
"""
for idx, row in missing_df.iterrows():
    md_content += f"| {row['Variable']} | {int(row['Missing_Count'])} |\n"

md_content += """
## 2. Summary Statistics (Cleaned Data)
""" + md_content + """

## 3. Visualizations
- Histograms of #volume_ml# and #Conc_B_ml# (imputed) are saved as #
    histograms_cleaned_data.png#.
"""

with open('/workspace/cleaning_report.md', 'w') as f:
    f.write(md_content)

print("Data cleaning and imputation completed successfully.")
print(f"Missing counts saved to: /workspace/cleaning_report.md")
print(f"Histograms saved to: /workspace/histograms_cleaned_data.png")
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Define paths
    input_path = '/inputs/chromatography_combined.csv'
```

```
output_path = '/workspace/quality_audit_report.md'

# Load data
df = pd.read_csv(input_path)
print(f"Debug: Columns in source CSV: {df.columns.tolist()}")
print(f"Debug: Data types of relevant columns:\n{df[['run_no', 'volume_ml', '
    UV_1_280_mAU']].dtypes}")

# Filter for specified variables
required_vars = ['run_no', 'Fraction_unique_ID', 'Conc_B_ml', 'UV_1_280_mAU',
    'volume_ml', 'chromatography_stage']
df = df[required_vars]

# Aggregate normalized peak areas by run_no on the full dataframe
df_agg = df.groupby('run_no').agg({
    'Fraction_unique_ID': 'count',
    'Conc_B_ml': 'mean',
    'volume_ml': 'sum'
}).reset_index()
df_agg.columns = ['run_no', 'total_fractions', 'mean_conc_B_ml', '
    run_total_volume_ml']

# Sort raw data for baseline calculation
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)
df['cum_vol'] = df['volume_ml'].cumsum()
# Calculate baseline threshold directly on df to avoid KeyError
df['baseline_threshold_ml'] = df.groupby('run_no')['volume_ml'].transform('sum
    ') * 0.10
df['is_baseline'] = df['cum_vol'] <= df['baseline_threshold_ml']

# Calculate run-level SNR using vectorized operations
df['noise'] = df.groupby('run_no')['UV_1_280_mAU'].transform(lambda x: x.loc[x
     <= df.loc[x.name, 'baseline_threshold_ml']].sum() if len(x) > 0 else 0)
df['signal'] = df.groupby('run_no')['UV_1_280_mAU'].transform(lambda x: x.loc[
    x > df.loc[x.name, 'baseline_threshold_ml']].sum() if len(x) > 0 else 0)
df['snr'] = df['signal'] / df['noise']
df['snr'] = df['snr'].replace([np.inf, -np.inf], np.nan)

# Calculate mean SNR directly on aggregated dataframe
df_agg['mean_snr'] = df.groupby('run_no')['snr'].mean()

# Calculate Proportion: Sum of UV in first 10% / Total UV directly on
    aggregated dataframe
# Calculate threshold volume for each run (10% of total volume)
df_agg['first_10_threshold'] = df_agg['run_total_volume_ml'] * 0.10

# Calculate total UV for each run on the raw dataframe grouped by run_no
df['total_uv'] = df.groupby('run_no')['UV_1_280_mAU'].transform('sum')
df_agg['total_uv'] = df.groupby('run_no')['UV_1_280_mAU'].transform('sum').
    groupby(level=0).sum()

# Calculate first 10% UV: Sum UV where volume_ml <= first_10_threshold for
    that run
# We need to map the threshold back to the raw df to filter correctly
df['first_10_threshold_raw'] = df.groupby('run_no')['total_uv'].transform('sum
    ') * 0.10
df['first_10_uv'] = df.groupby('run_no')['UV_1_280_mAU'].transform(
    lambda x: x.loc[x <= df.loc[x.name, 'first_10_threshold_raw']].sum() if
        len(x) > 0 else 0
```

```
    )
    df_agg['first_10_uv'] = df.groupby('run_no')['first_10_uv'].transform('sum').
        groupby(level=0).sum()

    # Avoid division by zero
    df_agg['prop_first_10'] = np.where(df_agg['total_uv'] != 0, df_agg['
        first_10_uv'] / df_agg['total_uv'], np.nan)

    # Identify runs with SNR < 3:1
    low_snr_runs = df_agg[df_agg['mean_snr'] < 3.0]

    # Generate Markdown
    md_content = "# Quality Audit and Proportion Comparison Report\n\n"
    md_content += "## Summary Table\n\n"
    md_content += "| run_no | total_fractions | mean_conc_B_ml | mean_snr | 
        prop_first_10 | \n"
    md_content += "
        |--------|-----------------|----------------|----------|---------------|\n"

    for _, row in df_agg.iterrows():
        md_content += f"| {row['run_no']} | {row['total_fractions']} | {row['
            mean_conc_B_ml']:.4f} | {row['mean_snr']:.4f} | {row['prop_first_10
            ']:.4f} |\n"

    md_content += "\n## Key Findings\n\n"

    if len(low_snr_runs) > 0:
        md_content += f"- **Runs with SNR < 3:1**: {len(low_snr_runs)} runs 
            identified.\n"
        md_content += "- **Specific Runs**: "
        run_list = ", ".join([str(r['run_no']) for _, r in low_snr_runs.iterrows()
            ])
        md_content += f"{run_list}\n"
    else:
        md_content += "- No runs identified with SNR < 3:1.\n"

    # Save file
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    with open(output_path, 'w') as f:
        f.write(md_content)

    print(f"Report saved to {output_path}")

if __name__ == "__main__":
    Code_Updates()
```

Listing 2: Code_3